Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «КиСБ»
Тема: «Избыточное кодирование данных в информационных системах. Код Хемминга»

Выполнил:
Студент 4 курса
Группы АС-65
Ракецкий П. П.
Проверил:
Хацкевич А. С.

Брест 2026

Цель: приобретение практических навыков кодирования\декодирования двоичных данных при использовании кода Хемминга.

Вариант 3

Задачи:

1. Закрепить теоретические знания по использованию итеративных кодов для повышения надежности передачи и хранения в памяти компьютера двоичных данных.
2. Разработать приложение для кодирования/декодирования двоичной информации итеративным кодом с различной относительной избыточностью кодовых слов.
3. Результаты выполнения лабораторной работы оформить в виде описания разработанного приложения, методики выполнения экспериментов с использованием приложения и результатов эксперимента.

Код программы:

```
import random

random.seed(12345)


def build_h(k, r):
    cols = []
    w = 2
    while len(cols) < k:
        for x in range(1, 1 << r):
            if bin(x).count('1') == w:
                cols.append(x)
        w += 1
    cols = cols[:k]
    P = [[(cols[j] >> (r - 1 - i)) & 1 for j in range(k)] for i in range(r)]
    H = [P[i] + [1 if j == i else 0 for j in range(r)] for i in range(r)]
    return H, r, cols


def add_parity_row(H, r, n):
    H2 = [row[:] + [0] for row in H]
    H2.append([1] * (n + 1))
    return H2, r + 1


def encode(Xk, H, r):
    return [sum(H[i][j] & Xk[j] for j in range(len(Xk))) % 2 for i in
range(r)]


def syndrome(Yn, H, r):
    return [sum(H[i][j] & Yn[j] for j in range(len(Yn))) % 2 for i in
range(r)]


def s_to_int(S):
    v = 0
    for b in S:
        v = (v << 1) | b
    return v


def cols_of(H, r):
    out = []
```

```
    for j in range(len(H[0])):
        v = 0
        for i in range(r):
            v = (v << 1) | H[i][j]
        if v != 0:
            out.append(v)
    return out


def decode(Yn, H, r, cols, verbose=True):
    S = syndrome(Yn, H, r)
    s_int = s_to_int(S)
    if verbose:
        print(f"    S = {''.join(map(str, S))} = {s_int}")
    if s_int == 0:
        if verbose: print("    ошибок нет")
        return Yn[:], S, None
    if s_int in cols:
        idx = cols.index(s_int)
        if verbose: print(f"    ошибка в позиции {idx + 1} (столбец
{s_int})")
        Yf = Yn[:]
        Yf[idx] ^= 1
        return Yf, S, idx + 1
    if verbose: print("    обнаружено, но не исправляется (S не столбец H)")
    return Yn[:], S, None


def part1():
    print("=" * 60)
    print(" ЧАСТЬ 1. КОД ХЭММИНГА (вариант 3)")
    print("=" * 60)

    Xk = [int(c) for c in "0001001100001100"]
    k = len(Xk)
    print(f"Xk ({k} бит) = {''.join(map(str, Xk))}")

    r = 5
    H, r, cols = build_h(k, r)
    n = k + r
    print(f"\n(21,16)-код: r={r}, n={n}, R={k/n:.4f}")
    print("Столбцы P':", cols)

    Xr = encode(Xk, H, r)
    Xn = Xk + Xr
    print(f"Xr = {''.join(map(str, Xr))}")
    print(f"Xn = {''.join(map(str, Xn))}  (n={len(Xn)})")
    print("H*Xn^T =", syndrome(Xn, H, r))

    print("\n[dmin=3] 0 ошибок:")
    decode(Xn, H, r, cols)

    print("[dmin=3] 1 ошибка (поз.5):")
    Y1 = Xn[:]; Y1[4] ^= 1
    Yf1, S1, p1 = decode(Y1, H, r, cols)
    print(f"    исправлено? {Yf1 == Xn}")

    print("[dmin=3] 2 ошибки (поз.5,9):")
    Y2 = Xn[:]; Y2[4] ^= 1; Y2[8] ^= 1
    Yf2, S2, p2 = decode(Y2, H, r, cols)
    print(f"    исправлено? {Yf2 == Xn}")

    print("\n--- Расширенный код (dmin=4) ---")
    H4, r4 = add_parity_row(H, r, n)
```

```
    cols4 = cols_of(H4, r4)
    pbit = sum(Xn) % 2
    Xn4 = Xn + [pbit]
    print(f"Xn4 = {''.join(map(str, Xn4))}  (n={len(Xn4)},
R={k/len(Xn4):.4f})")
    print("H4*Xn4^T =", syndrome(Xn4, H4, r4))

    print("[dmin=4] 1 ошибка (поз.5):")
    Y41 = Xn4[:]; Y41[4] ^= 1
    Yf41, _, _ = decode(Y41, H4, r4, cols4)
    print(f"    исправлено? {Yf41 == Xn4}")

    print("[dmin=4] 2 ошибки (поз.5,9):")
    Y42 = Xn4[:]; Y42[4] ^= 1; Y42[8] ^= 1
    Yf42, S42, _ = decode(Y42, H4, r4, cols4)
    print(f"    S = {s_to_int(S42)}; исправлено? {Yf42 == Xn4}")


def to_grid(bits, rows, cols):
    assert len(bits) == rows * cols
    return [bits[i*cols:(i+1)*cols] for i in range(rows)]


def from_grid(g):
    return [b for row in g for b in row]


def iter_code_2d(Xk, rows, cols):
    g = to_grid(Xk, rows, cols)
    h = [sum(row) % 2 for row in g]
    v = [sum(g[i][j] for i in range(rows)) % 2 for j in range(cols)]
    Xr = h + v
    Xn = Xk + Xr
    return Xn, dict(rows=rows, cols=cols, g=g, h=h, v=v,
                    info_len=len(Xk), total=len(Xn))


def iter_code_3d(Xk, rows, cols, kind="diag"):
    Xn2, m = iter_code_2d(Xk, rows, cols)
    g = m["g"]
    if kind == "diag":
        d = [g[i][i] for i in range(min(rows, cols))]
        dpar = sum(d) % 2
        Xr_extra = [dpar]
    else:
        d = [g[i][cols-1-i] for i in range(min(rows, cols))]
        dpar = sum(d) % 2
        Xr_extra = [dpar]
    Xn = Xn2 + Xr_extra
    m = dict(m); m["total"] = len(Xn); m["extra"] = Xr_extra
    return Xn, m


def iter_code_4d(Xk, rows, cols):
    Xn3, m = iter_code_3d(Xk, rows, cols, "diag")
    g = m["g"]
    anti = [g[i][cols-1-i] for i in range(min(rows, cols))]
    apar = sum(anti) % 2
    Xn = Xn3 + [apar]
    m = dict(m); m["total"] = len(Xn); m["anti"] = apar
    return Xn, m


def iter_code_5d(Xk, rows, cols):
```

```
    Xn4, m = iter_code_4d(Xk, rows, cols)
    allpar = sum(Xk) % 2
    Xn = Xn4 + [allpar]
    m = dict(m); m["total"] = len(Xn); m["allpar"] = allpar
    return Xn, m


def iter_encode(Xk, rows, cols, groups):
    if groups == 2: return iter_code_2d(Xk, rows, cols)
    if groups == 3: return iter_code_3d(Xk, rows, cols)
    if groups == 4: return iter_code_4d(Xk, rows, cols)
    if groups == 5: return iter_code_5d(Xk, rows, cols)
    raise ValueError


def iter_decode_2d(Yn, rows, cols):
    n_info = rows * cols
    Yk = Yn[:n_info]
    Yh = Yn[n_info:n_info+rows]
    Yv = Yn[n_info+rows:n_info+rows+cols]
    g = to_grid(Yk, rows, cols)
    h_calc = [sum(row) % 2 for row in g]
    v_calc = [sum(g[i][j] for i in range(rows)) % 2 for j in range(cols)]
    bad_r = [i for i in range(rows) if h_calc[i] != Yh[i]]
    bad_c = [j for j in range(cols) if v_calc[j] != Yv[j]]
    if len(bad_r) == 0 and len(bad_c) == 0:
        return Yn[:], Yk, "нет ошибок"
    if len(bad_r) == 1 and len(bad_c) == 1:
        i, j = bad_r[0], bad_c[0]
        g[i][j] ^= 1
        return from_grid(g) + Yn[n_info:], from_grid(g), f"исправлена позиция
({i},{j})"
    return Yn[:], Yk, "многократная ошибка, не исправлено"


def eval_correcting(Xk, rows, cols, groups, n_trials=200, max_err=3):
    Xn, _ = iter_encode(Xk, rows, cols, groups)
    res = {}
    for err in range(1, max_err + 1):
        N1 = N2 = N3 = 0
        for _ in range(n_trials):
            pos = random.sample(range(rows * cols), err)
            Y = Xn[:]
            for p in pos: Y[p] ^= 1
            N1 += 1
            Yf, Yk_fix, status = iter_decode_2d(Y, rows, cols)
            if "исправлена" in status or "нет" in status:
                N2 += 1
            if Yf[:len(Xn)] == Xn:
                N3 += 1
        res[err] = (N1, N2, N3)
    return res


def part2():
    print("\n" + "=" * 60)
    print(" ЧАСТЬ 2. ИТЕРАТИВНЫЕ КОДЫ (вариант 3)")
    print("=" * 60)

    Xk = [int(c) for c in "101100111000111100001010"]
    print("Xk (24 бита) =", ''.join(map(str, Xk)))
    print("Проверка длины:", len(Xk))

    configs = [
```

```
        (4, 6, 2), (4, 6, 3), (4, 6, 4), (4, 6, 5),
        (3, 8, 2), (3, 8, 3), (3, 8, 4), (3, 8, 5),
    ]
    for (r_, c_, g_) in configs:
        Xn, m = iter_encode(Xk, r_, c_, g_)
        print(f"\n--- {r_}x{c_}, групп паритетов={g_} ---")
        print(f"Xn ({len(Xn)} бит) = {''.join(map(str, Xn))}")
        print(f"R = {len(Xk)/len(Xn):.4f}, избыточность = {len(Xn)-len(Xk)}
бит")
        if g_ == 2:
            Y = Xn[:]; Y[random.randrange(r_ * c_)] ^= 1
            Yf, Yk_fix, status = iter_decode_2d(Y, r_, c_)
            print("  ошибка:", status, "| восстановлено?", Yf[:len(Xn)] ==
Xn)

    print("\n--- Оценка N2/N1 и N3/N1 (4x6, 2 группы, 200 испытаний) ---")
    res = eval_correcting(Xk, 4, 6, 2, n_trials=200, max_err=3)
    for err, (N1, N2, N3) in res.items():
        print(f"  кратность {err}: N2/N1 = {N2/N1:.3f}, N3/N1 = {N3/N1:.3f}")


if __name__ == "__main__":
    part1()
    part2()
```

Вывод программы в консоле:

==========

ЧАСТЬ 1. КОД ХЭММИНГА (вариант 3)

==========

Xk (16 бит) = 0001001100001100

(21,16)-код: r=5, n=21, R=0.7619

Столбцы P': [3, 5, 6, 9, 10, 12, 17, 18, 20, 24, 7, 11, 13, 14, 19, 21]

Xr = 01001

Xn = 000100110000110001001 (n=21)

H*Xn^T = [0, 0, 0, 0, 0]

[dmin=3] 0 ошибок:

S = 00000 = 0

ошибок нет

[dmin=3] 1 ошибка (поз.5):

S = 01010 = 10

ошибка в позиции 5 (столбец 10)

исправлено? True

[dmin=3] 2 ошибки (поз.5,9):

S = 11110 = 30

обнаружено, но не исправляется (S не столбец H)

исправлено? False

--- Расширенный код (dmin=4) ---

Xn4 = 0001001100001100010011 (n=22, R=0.7273)

H4*Xn4^T = [0, 0, 0, 0, 0, 0]

[dmin=4] 1 ошибка (поз.5):

S = 010101 = 21

ошибка в позиции 5 (столбец 21)

исправлено? True

[dmin=4] 2 ошибки (поз.5,9):

S = 111100 = 60

обнаружено, но не исправляется (S не столбец H)

S = 60; исправлено? False

==========

ЧАСТЬ 2. ИТЕРАТИВНЫЕ КОДЫ (вариант 3)

==========

Xk (24 бита) = 101100111000111100001010

Проверка длины: 24

--- 4x6, групп паритетов=2 ---

Xn (34 бит) = 1011001110001111000010101100100010

R = 0.7059, избыточность = 10 бит

ошибка: исправлена позиция (2,1) | восстановлено? True

--- 4x6, групп паритетов=3 ---

Xn (35 бит) = 10110011100011110000101011001000101

R = 0.6857, избыточность = 11 бит

--- 4x6, групп паритетов=4 ---

Xn (36 бит) = 101100111000111100001010110010001010

R = 0.6667, избыточность = 12 бит

--- 4x6, групп паритетов=5 ---

Xn (37 бит) = 1011001110001111000010101100100010100

R = 0.6486, избыточность = 13 бит

--- 3x8, групп паритетов=2 ---

Xn (35 бит) = 10110011100011110000101011000110110

R = 0.6857, избыточность = 11 бит

ошибка: исправлена позиция (2,7) | восстановлено? True

--- 3x8, групп паритетов=3 ---

Xn (36 бит) = 101100111000111100001010110001101101

R = 0.6667, избыточность = 12 бит

--- 3x8, групп паритетов=4 ---

Xn (37 бит) = 1011001110001111000010101100011011010

R = 0.6486, избыточность = 13 бит

--- 3x8, групп паритетов=5 ---

Xn (38 бит) = 10110011100011110000101011000110110100

R = 0.6316, избыточность = 14 бит

--- Оценка N2/N1 и N3/N1 (4x6, 2 группы, 200 испытаний) ---

кратность 1: N2/N1 = 1.000, N3/N1 = 1.000

кратность 2: N2/N1 = 0.000, N3/N1 = 0.000

кратность 3: N2/N1 = 0.165, N3/N1 = 0.000

Вывод: в работе реализованы код Хэмминга с dmin=3 и расширенный dmin=4, а также итеративные коды для матриц 4×6 и 3×8 с 2–5 группами паритетов. Код Хэмминга dmin=3 исправляет одиночную ошибку и обнаруживает двойную, dmin=4 исправляет одну и обнаруживает две. Итеративный 2D-код с двумя группами паритетов (dmin=4) исправляет все одиночные ошибки в информационной части (N2/N1=N3/N1=1.000), но не исправляет кратные; при кратности 3 возможны ложные срабатывания (N2/N1=0.165, N3/N1=0) из-за маскировки ошибок под одиночную. Увеличение числа групп паритетов повышает избыточность и снижает скорость кода. Цель работы достигнута.